JAWAHAR NAVODAYA VIDYALAYA:: MADDIRALA,PALNADU

UNIT-3

Structured Query Language (SQL)

Introduction to Structured Query Language (SQL)

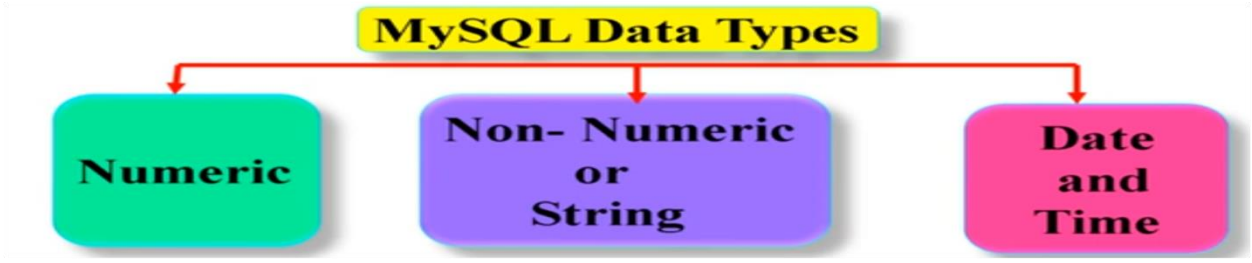
SQL: Database management systems(DBMS)has special kind of programming language called query language that can be used to access data from the database. The Structured Query Language (SQL) is the most popular query language used by major RDBMS such as **MySQL, ORACLE, SQL Server**, etc.

SQL is easy to learn as the statements comprise of descriptive English words and are not case sensitive.

MySQL: MySQL is open source RDBMS software, which canbe easily downloaded from the official website https://dev.mysql.com/downloads.



Data Types in MySQL:Data type indicates the type of data value that an attribute can have.MySQL Support **3** categories of data types namely 1.numeric types, 2.string (character and byte) types 3.date and time types, and.

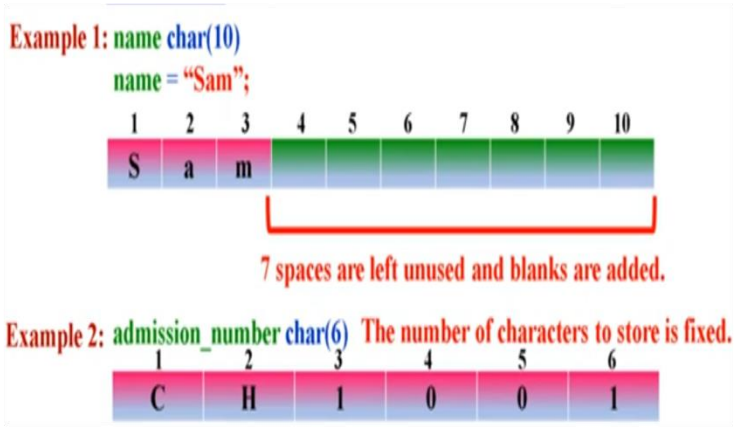


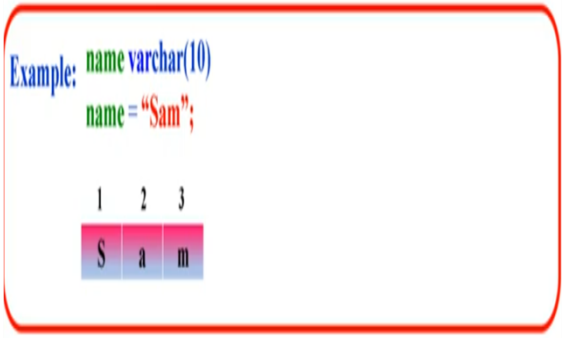
Commonly used data types in MySQL

Data type	**Description**
INT	INT specifies an integer value. Each INT value occupies 4 bytes of storage
FLOAT (DECIMAL)	Holds numbers with decimal points. Each FLOAT value occupies 4 bytes.
CHAR	Specifies character type data of length n where n could be any value from 0 to255. **CHAR is of fixed length, default is 1**
VARCHAR	Specifies character type data of length 'n' where n could be any value from 0to 65535.**VARCHAR is a variable-length data type**
DATE	The DATE type is used for dates in 'YYYY-MM-DD' format. YYYY is the 4 digityear, MM is the 2 digit month and DD is the 2 digit date

DIFFERENCE BETWEEN CHAR AND VARCHAR

CHAR	VARCHAR
1.Char stands for**CHARACTER**	1.Varchar stands for**VARIABLE CHARACTER**
2.Char is a **Fixed** length	2.Varchar is a **Variable** length
3.Char adds blanks when a value entered is shorter than its length, Wastage of Memory is high	3.No blanks are added ,if the length is shorter than its length, It does not waste the memory
4.Ex: name char(10) If data is stored in NAME column as **"SAM"** It will occupy memory space of 10 characters even if ,we are having less no.ofcharacters in text. S A M	4.Ex: name varchar(10) If data is stored in NAME column as **SAM** It will occupy space of 3 characters only , remaining 7 characters it is used for other memory purpose. S A M
5.Mostly used for PINCODE, GENDER, PIHONE NUMBER, BOOLEAN,ADM.NO	5.Mostyl used for names, Long characters
6.It is Fast	6.It is Slow





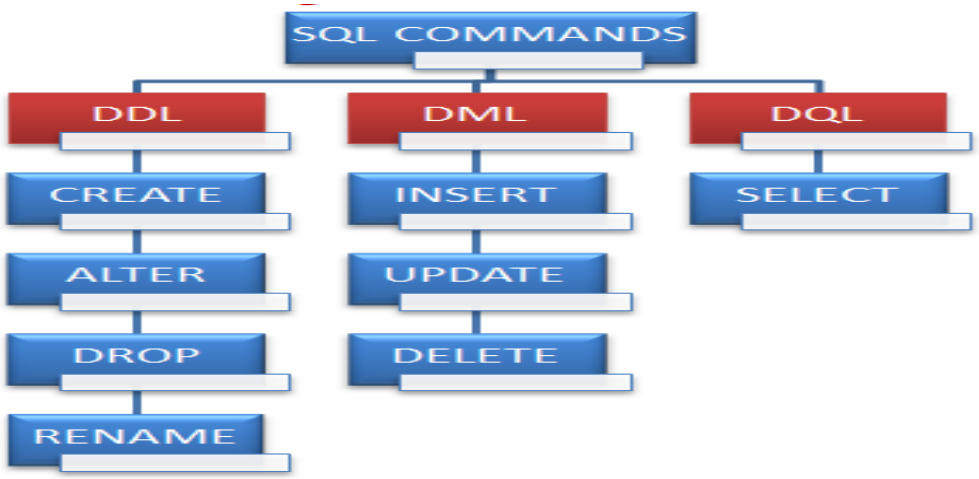
Create table using data types

```
mysql>create table data(admno char(6),sname varchar(20),rno int,percent float,dob date);
mysql> insert into data values(001234,'A',1,80.25,'2010-10-10');
mysql> insert into data values(001235,'B',1,90.50,'2011-11-11');
mysql> insert into data values(001236,'C',3,50.02,'2009-09-09');
mysql>select * from data;
```

admno	sname	rno	percent	dob
001234	A	1	80.25	2010-10-10
001235	B	2	90.5	2011-11-11
001236	C	3	50.45	2009-09-09

```
mysql> insert into data values('0012367','D',3,90.45,'2009-09-09');
ERROR 1406 (22001): Data too long for column 'admno' at row 1
mysql> insert into data values('001237','bhanu gari veera simha reddy ',3,90.45,'2009-09-09');
ERROR 1406 (22001): Data too long for column 'sname' at row 1
mysql> insert into data values('001236','D','e',90.45,'2009-09-09');
ERROR 1366 (HY000): Incorrect integer value: 'e' for column 'rno' at row 1
```

SQL COMMANDS



1.DDL(Data Definition Language):DDL allows you to perform tasks related to the **structure** of the a table. DDL supports 4 types of commands

a. Create b. Alter c. Drop d. Rename

a.**Create command**: Data definition starts with the create statement. This statement is used to create a database and tables
Syntax:CREATE DATABASE databasename;
CREATE TABLE tablename(Col_namedatatype constraint……)

Example: create database jnv;
create table jnvk(snoint not nul,,,,,,) ;

b.Alter Command: After creating a table we may realize that we need to add/remove an attribute or to modify the datatype of an existing attribute or to add constraint in attribute. In all such cases, we need to change or alter the structure of the table by using the alter statement.

Command	Purpose	Affects	Example
ALTER	Changes table structure	Table design	`ALTER TABLE student ADD age INT;`
UPDATE	Changes table data	Data (rows)	`UPDATE student SET marks=95 WHERE roll_no=101;`

Syntax:**ALTER TABLE tablename ADD/DROP/MODIFY/CHANGE/RENAME** attribute1,attribute2,..

Note : RENAME →ONLY RENAME
MODIFY → ONLY DATATYPE
CHANGE→ CHANGE RENAME + DATATYPE



DDL ALTER COMMAND

0.CREATE TABLE DEMO

```
mysql> create table demo(
    -> rno int,
    -> sname varchar(30)
    -> );
Query OK, 0 rows affected (0.05 sec)
```

1.ADD NEW COLUMN IN TABLE USING ALTER

```
mysql> alter table demo add dob date;
Query OK, 0 rows affected (0.04 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

2.DROP COLUMN IN TABLE USING ALTER

```
mysql> alter table demo drop dob;
Query OK, 0 rows affected (0.04 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

3.ADD PRIMARY KEY in demo TABLE

```
mysql> alter table demo add primary key(rno);
Query OK, 0 rows affected (0.07 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

4.MODIFY DATA TYPE in demo TABLE

```
mysql> ALTER TABLE DEMO MODIFY RNO VARCHAR(8);
Query OK, 0 rows affected (0.07 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

5.CHANGE COLUMN NAME in demo TABLE

```
mysql> alter table demo change sname speru varchar(30);
Query OK, 0 rows affected (0.04 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

6. RENAME COLUMN NAME in demo TABLE

```
mysql> alter table demo rename column speru to sname;
Query OK, 0 rows affected (0.04 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

c. Drop Command: We can use DROP statement to remove a database or a table permanently from the system. However, one should be very cautious while using this statement as it cannot be undone.

Syntax to drop a database:DROP DATABASE database_name;
Ex: DROP DATABASE jnv;

Syntax to drop a table:DROP TABLE table_name;
Ex: DROP TABLE jnvk;

D.Rename Command:is used to rename an existing **database or table**. You can rename a table using the RENAME TABLE command and a database using the RENAME DATABASE command.

Syntax: RENAME TABLE old_table_name TO new_table_name;
Ex: rename table jnvk to jnvs

8.5 SQL for Data Manipulation

2.DML(Data Manipulation Language):DML is used for Inserting, Updating , and deleting the data in the table. DML supports 3 types of commands

a. Insert **b. Update** **c. Delete**

insert command types in mysql with examples

a.**Insert Command :** In MySQL, the **INSERT** command is used to add new records into a table.

Syntax:

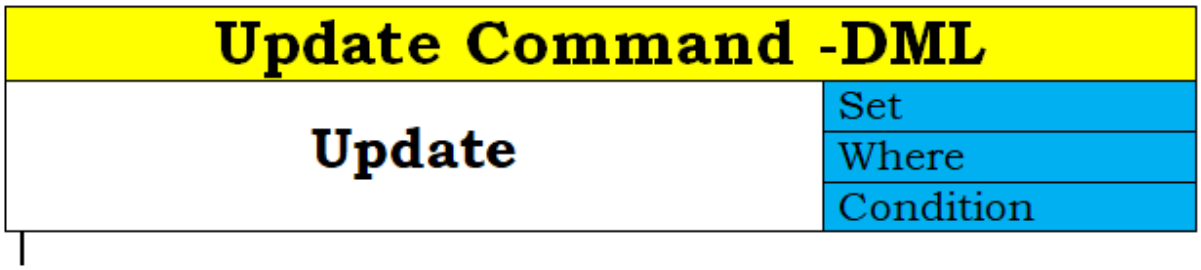
1.INSERT INTO table_name (columns) VALUES (values);
Ex:INSERT INTO students (roll_no, first_name, last_name, dob, class, marks) VALUES (101, 'Syed', 'Rafiq', '2004-02-15', 'XI', 85);

2.INSERT INTO table_name VALUES (values);
Ex:INSERT INTO students VALUES (102, 'Syed', 'Rafiq', '2003-03-20', 'XI', 85);

3.Inserting Multiple Rows:

INSERT INTO students (roll_no, first_name, last_name, dob, class, marks) VALUES (103, 'Allu', 'Arjun', '2003-11-10', 'XI', 92), (104, 'Mahesh', 'Babu', '2004-07-17', 'X', 78);

b.Update Command : We may need to make changes in the value(s) of one or more columns of existing records in a **table** for this we use UPDATE command.



DML ALTER COMMAND

0.insert the values in demo table

```
mysql> insert into demo values(1,'a','2010-10-10',10),(2,'b','2011-11-11',20),(3,'c','2009-09-09',30);
Query OK, 3 rows affected (0.03 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> select*from demo;
+-----+-------+------------+-------+
| SNO | sname | dob        | marks |
+-----+-------+------------+-------+
| 1   | a     | 2010-10-10 |    10 |
| 2   | b     | 2011-11-11 |    20 |
| 3   | c     | 2009-09-09 |    30 |
+-----+-------+------------+-------+
```

1.UPDATE THE VALUES USING SET

```
mysql> update demo set sname='rafiq' where sno=1;
Query OK, 1 row affected (0.02 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> update demo set sname='syed' where sno=2;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> update demo set sname='sam' where sno=3;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

2.UPDATE THE VALUES USING WHERE CONDITION

```
mysql> update demo set marks=marks+40 where sno=1;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

C.Delete Command: The DELETE statement is used to delete one or more record(s) from a **table.**

Syntax:DELETE FROM table_name WHERE condition;

Ex:DELETE FROM STUDENT WHERE RollNumber = 2;

8.6 SQL for Data Query

III.DQL(Data Query Language)-Select: The SQL statement **SELECT** is used to retrieve data from the tables in a database and the output is also displayed in tabular form.

Syntax:SELECT column1, column2, ... FROM table_name WHERE condition

Example :

Step 1 :First create **DEPT** table;

CREATE TABLE **DEPT**(DeptId VARCHAR(50) PRIMARY KEY,DeptName VARCHAR(255) NOT NULL,Location VARCHAR(255) NOT NULL);

```
mysql> desc dept;
+----------+--------------+------+-----+---------+-------+
| Field    | Type         | Null | Key | Default | Extra |
+----------+--------------+------+-----+---------+-------+
| DeptId   | varchar(50)  | NO   | PRI | NULL    |       |
| DeptName | varchar(255) | NO   |     | NULL    |       |
| Location | varchar(255) | NO   |     | NULL    |       |
+----------+--------------+------+-----+---------+-------+
```

```
mysql> insert into dept values('D01','IT','KNL');
Query OK, 1 row affected (0.05 sec)

mysql> insert into dept values('D02','hr','KDP');
Query OK, 1 row affected (0.01 sec)

mysql> insert into dept values('D03','FINANCE','HYD');
Query OK, 1 row affected (0.05 sec)

mysql> insert into dept values('D04','MARKETING','CHENNAI');
Query OK, 1 row affected (0.05 sec)

mysql> insert into dept values('D05','ADMIN','BLR');
Query OK, 1 row affected (0.05 sec)
```

Step 2 Second create **EMP** table by giving foreign key reference

CREATE TABLE **EMP(**

EmpNo INT PRIMARY KEY,

Ename VARCHAR(255) NOT NULL,

Salary int NOT NULL,

Bonus int not null,

DeptId VARCHAR(50),

FOREIGN KEY (DeptId) REFERENCES DEPT(DeptId));

mysql>insert into emp values(101,'Aaliya',10000,234,'D02');

```
mysql> select* from emp;
+-------+----------+--------+-------+--------+
| EmpNo | Ename    | Salary | Bonus | DeptId |
+-------+----------+--------+-------+--------+
|   101 | Aaliya   |  10000 |   234 | D02    |
|   102 | Krithika |  60000 |   123 | D01    |
|   103 | Shabbir  |  45000 |   566 | D01    |
|   104 | Gurpreet |  19000 |   565 | D04    |
|   105 | Joseph   |  34000 |   875 | D03    |
|   106 | Sanya    |  48000 |   695 | D02    |
|   110 | Tanya    |  50000 |   467 | D05    |
+-------+----------+--------+-------+--------+
7 rows in set (0.00 sec)
```

Inserting NULL Values

```
mysql> ALTER TABLE EMP MODIFY BONUS INT NULL;
```

```
mysql> insert into emp values(109,'Daribha',42000,null,'D04');
```

```
mysql> select* from emp;
+-------+----------+--------+-------+--------+
| EmpNo | Ename    | Salary | BONUS | DeptId |
+-------+----------+--------+-------+--------+
|   101 | Aaliya   |  10000 |   234 | D02    |
|   102 | Krithika |  60000 |   123 | D01    |
|   103 | Shabbir  |  45000 |   566 | D01    |
|   104 | Gurpreet |  19000 |   565 | D04    |
|   105 | Joseph   |  34000 |   875 | D03    |
|   106 | Sanya    |  48000 |   695 | D02    |
|   107 | Vergese  |  15000 |  NULL | D01    |
|   108 | Nachaobi |  29000 |  NULL | D05    |
|   109 | Daribha  |  42000 |  NULL | D04    |
|   110 | Tanya    |  50000 |   467 | D05    |
+-------+----------+--------+-------+--------+
10 rows in set (0.00 sec)
```

EmpNo	Ename	Salary	Bonus	DeptId
101	Aaliya	10000	234	D02
102	Kritika	60000	123	D01
103	Shabbir	45000	566	D01
104	Gurpreet	19000	565	D04
105	Joseph	34000	875	D03
106	Sanya	48000	695	D02
107	Vergese	15000		D01
108	Nachaobi	29000		D05
109	Daribha	42000		D04
110	Tanya	50000	467	D05

8.6.2 QUERYING using Database OFFICE

A.Retrieve selected columns

```
mysql>select empno from emp;
mysql>select empno,ename from emp;
```

B.Renaming of columns:To rename a column we use "AS"

```
mysql> select ename as name from emp;
mysql> select ename as peru from emp;
mysql> select ename as name ,salary*12 from emp;
mysql> select ename as name,salary*12 as 'Annual Salary' from emp;
```

C.DistinctClause: The statement when combined with SELECT DISTINCT clause, returns records without repetition (distinct records).

Syntax:select DISTINCT <column name> from <table name>

```
mysql> select distinct deptid from emp;
```

D.WHERE clause: The WHERE clause is used to retrieve data that meet some specified conditions

```
mysql> select DISTINCT salary from EMP WHERE deptid='D01';
```

RELATIONAL OPERATORS in MySQL:->,>=,<,<=,!=,=

Query :Display all the employees who are earning more than 5000 and work in department with DeptId D04

```
mysql>select * from emp WHERE salary > 5000 AND Deptid='D04';
```

Query :display name and department number of all those employees who are earning salary between 20000 and 50000

```
mysql>select ename,deptid from emp where salary >=20000 and salary <=50000;
```

or

```
mysql>select ename,deptid from emp where salary between 20000 and 50000;
```

display details of all the employees who are working either in DeptId D01, D02 or D04.

```
mysql>select * from emp WHERE salary < 55000 AND Deptid='D01';
mysql> select ename,empno from emp where deptid!='D01';
```

Query :display records of all the employees except Aaliya.

```
mysql>select*from emp where not ename='Aaliya';
```

```
CREATE TABLE ros ( sno INT PRIMARY KEY, sname VARCHAR(20),
  city VARCHAR(10),   house VARCHAR(20),   class VARCHAR(5), marks INT);
```

```
mysql> select*from ros;
+-----+---------+------+----------+-------+-------+
| sno | sname   | city | house    | class | marks |
+-----+---------+------+----------+-------+-------+
|   1 | arjun   | knl  | nilagiri | ix    |    85 |
|   2 | bunny   | ndl  | shivalik | ix    |    75 |
|   3 | durga   | atp  | aravali  | ix    |    64 |
|   4 | eswar   | ong  | udaygiri | ix    |    51 |
|   5 | raj     | knl  | nilagiri | x     |    89 |
|   6 | gopal   | ndl  | shivalik | x     |    92 |
|   7 | harish  | atp  | aravali  | x     |    75 |
|   8 | saif    | chi  | udaygiri | x     |    96 |
|   9 | teja    | chi  | udaygiri | xi    |    46 |
|  10 | ram     | knl  | nilagiri | xii   |    67 |
+-----+---------+------+----------+-------+-------+
```

```
mysql>select * from ros where marks >90;mysql> select * from ros where marks <90;
mysql> select * from ros where marks <=90;
mysql> select * from ros where marks >=90;
mysql> select * from ros where city!='knl';mysql> select * from ros where city<>'knl';
mysql> select * from ros where city='knl';
mysql>select * from ros where house<>'nilagiri';
mysql> select * from ros where house='nilagiri';
```

mysql> SELECT * FROM ROS WHERE CITY='CHI' AND HOUSE='UDAYGIRI';
mysql> SELECT * FROM ROS WHERE CITY='CHI' && HOUSE='UDAYGIRI';
mysql> SELECT * FROM ROS WHERE CITY='KNL' && CLASS='IX' && HOUSE='NILAGIRI';
SELECT * FROM ROS WHERE HOUSE='ARAVALI' OR HOUSE='UDAYGIRI';
mysql> SELECT * FROM ROS WHERE HOUSE='ARAVALI' OR CITY='CHI';
mysql> SELECT * FROM ROS WHERE HOUSE='ARAVALI' || CITY='CHI';
mysql> SELECT * FROM ROS WHERE NOT HOUSE='NILAGIRI';
mysql> SELECT * FROM ROS WHERE HOUSE!='NILAGIRI';

LOGICAL OPERATORS in MySQL:-

Logical Operator	Symbol
And	&&
Or	||
Not	!

mysql> select empno,ename from emp where deptid='D01' **AND** salary >= 50000;

mysql> select * from emp where deptid='D01' && salary >= 45000;

mysql> select*from emp where bonus is null **OR** salary <20000;

mysql>select *from emp where deptid='D05' **OR** salary >50000;

mysql> select * from emp where deptid='D01' || salary > 50000;

mysql> select*from emp wheredeptid='D01' or deptid='D02' **OR**deptid='D04';

mysql> select*from emp where bonus is null;

mysql> select*from emp where bonus is **NOT** null;

mysql> select*from emp where **NOT**deptid='D05';

mysql> select*from emp where NOT ename='Aaliya';

mysql>select ename,deptid,salary from emp where salary between 20000 AND 50000;

mysql> select*from emp where deptid='D01' or deptid='D02' or deptid='D04';

E.MEMBERSHIP OPERATOR(IN,NOT IN): The `IN` operator compares a value with a set of values and returns true if the value belongs to that set.
mysql> select*from emp where deptid **IN**('D01','D02','D04');
mysql> select*from emp where deptid **NOT IN**('D01','D02','D04');

F.ORDER BY:is used to display data in an ordered, By default, ORDER BY displays records in **ascending order**.To display the records in descending order, the **DESC** (means descending) keyword needs to be written with that column.
mysql> select*from emp **ORDER BY** salary;

Query :Display details of all the employees in ascending order of their salaries.

mysql> select*from emp **ORDER BY** salary**ASC**;

Query :Display details of all the employees in decending order of their salaries

mysql> select*from emp **ORDER BY** salary **DESC**;

(G) Handling NULL Values: SQL supports a special value called **NULL** to represent a missing or unknown value.

```
mysql> desc emp;
+--------+--------------+------+-----+---------+-------+
| Field  | Type         | Null | Key | Default | Extra |
+--------+--------------+------+-----+---------+-------+
| EmpNo  | int          | NO   | PRI | NULL    |       |
| Ename  | varchar(255) | NO   |     | NULL    |       |
| Salary | int          | NO   |     | NULL    |       |
| bonus  | int          | YES  |     | NULL    |       |
| DeptId | varchar(50)  | YES  | MUL | NULL    |       |
+--------+--------------+------+-----+---------+-------+
5 rows in set (0.03 sec)
```

Mysql>alter table emp modify bonus int null;

```
mysql> ALTER TABLE EMP MODIFY BONUS INT NULL;
```

mysql> select * from emp where bonus **IS NULL**;
mysql> select empno,ename from emp where bonus **IS NOT NULL**;

(H) Substring pattern matching (LIKE Operator):

SQL provides**LIKE** operator that can be used with clause to search WHERE for a specified pattern in a column.

The **LIKE** operator makes use of the following **two** wild card characters:

1.% (percent)— used to represent zero, one, or multiple characters .

Ex:Display details of all those employees whose name starts with 'A'.

mysql> select * from emp where ename like 'a%';

Ex:Displays details of all those employees whose name ends with 'a'.

mysql> select * from emp where ename like '%a';

Query :Display details of all those employees whose name starts with 'K'.

mysql> select*from emp where ename like 'K%';

select*from emp where ename like '%i%'; //display names containing “i”

2. _ (underscore)— used to represent a single character

Displays details of all those employees whose name consists of exactly 5 letters and starts with any letter but has ‘ANYA’ after that

mysql> select*from emp where ename like '_ANYA';

mysql> select * from emp where ename like '_____';

```
mysql> select*from emp where ename like "__l%";
+-------+---------+---------+-------+---------+
| EmpNo | Ename   | Salary  | BONUS | DeptId  |
+-------+---------+---------+-------+---------+
|   101 | Aaliya  |   10000 |   234 | D02     |
+-------+---------+---------+-------+---------+
1 row in set (0.00 sec)

mysql> select*from emp where ename like "__r%";
+-------+----------+---------+-------+---------+
| EmpNo | Ename    | Salary  | BONUS | DeptId  |
+-------+----------+---------+-------+---------+
|   104 | Gurpreet |   19000 |   565 | D04     |
|   107 | Vergese  |   15000 |  NULL | D01     |
|   109 | Daribha  |   42000 |  NULL | D04     |
+-------+----------+---------+-------+---------+
3 rows in set (0.00 sec)
```

mysql> SELECT * FROM Emp WHERE Ename LIKE 'D_____';